



ASSIGNMENT 3.2

Modulo local de inferencia

Convertir el lienzo en un comando reproducible

DURACION	MODALIDAD	PREREQUISITOS
2 horas	Parejas, learning by doing	Assignment 3.1 y starter con tests

1. Encargo

Implementar un modulo local que lea un CSV, valide cada fila, preprocese en el orden entrenado, cargue el modelo ya entrenado y escriba predicciones sin dejar una salida parcial si falla una fila.

Regla de continuidad

Trabaja exclusivamente en problem/starter; la carpeta solutions es referencia del debrief.

2. Punto de partida

- El `joblib` ya contiene el estimador, feature_names y model_version; no hay que reentrenarlo.
- Las pruebas describen la interfaz publica: contratos, carga, inferencia y CLI.
- La implementacion debe dejar preparado el bundle que S4 empaquetara.

3. Material que recibes

- problem/starter/ con pyproject.toml, tests y TODOs
- models/wine_quality_classifier.joblib entregado por el docente
- assets/inference_samples.csv con cinco muestras
- Lienzo de contrato de la clase 1

4. Tareas

01 Lee la especificacion - Ejecuta la suite roja y convierte cada nombre de test en una regla de la API local.

02 Implementa contratos - Completa request y prediction con Pydantic; rechaza extras, tipos imposibles y valores fuera de rango.

03 Implementa preprocesado - Construye un vector estable con las once features y una version explicita; no mezcles sample_id.

04 Implementa carga e inferencia - Valida el payload del artefacto antes de invocar predict/predict_proba y valida la salida resultante.

05 Completa el CLI - Lee todas las filas, acumula predicciones y escribe el CSV solo cuando el conjunto es valido.

06 Amplia los errores - Prueba columna extra, segunda fila invalida y artefacto incompatible; confirma que no queda CSV parcial.

5. Entrega

Evidencia	Que debe permitir comprobar
Codigo del starter	contracts.py, preprocess.py, inference.py y predict_file.py completados.
Tests	Suite en verde, incluyendo casos de error y ausencia de salida parcial.
Comando	CSV de cinco filas con sample_id, categoria, confianza y versiones.
Nota tecnica	Responsabilidad de cada modulo y dos decisiones de diseño.

6. Criterios de aceptacion

- [] uv run pytest termina sin fallos.
- [] El cargador comprueba feature_names y la interfaz minima del estimador.
- [] El vector respeta el orden del entrenamiento y excluye sample_id.
- [] Una fila invalida no deja un fichero de salida parcial.
- [] El CSV de salida contiene sample_id, quality_band, confidence, model_version y preprocessing_version.

7. Rubrica orientativa

Criterio	Puntos
Contratos y validacion	2
Preprocesado e inferencia	3
CLI y atomicidad	3
Tests y explicacion	2

8. Comprobacion

```
cd exercises/01-local-inference/01.02-wine-quality-inference-module/problem/starter
```

```
uv sync
```

```
uv run pytest
```

```
uv run ruff check src tests
```

```
uv run ruff format --check src tests
```

```
uv run python -m model_inference.predict_file --input assets/inference_samples.csv --output .tmp/wine_predictions.csv
```

9. Notas y limites

- No leas CSV desde preprocess.py ni cargues joblib desde el CLI.
- No subas el artefacto binario al repositorio.
- Si el modelo real no esta disponible, usa los clasificadores pequeños de los tests para comprobar la interfaz.

Referencia en el repositorio

semana3/modules/03-inference-contracts/exercises/01-local-inference/01.02-wine-quality-inference-module/problem/readme.md
semana3/modules/03-inference-contracts/guides/class-2-workshop.md